Syrian Private University

Faculty of Artificial Intelligence Engineering

Department of Systems Security and Computer Networks

Secure Chat-Based Environment Using AI-Based IDS

Prepared by:

William Rateb Albeni

Obaida Fattouh

Supervised by:

Dr. Wassim Aljuneidi

Eng. Abdulrahman Arar

Eng. Nagham Alashqar

Year

2026-2027

Table of Contents

Introduction 4

1-1. Preface 5

1-2. Scientific Problem and Project Justifications 5

1-3. Project Objectives 6

1-4. Previous Studies and State-of-the-Art 6

1-5. Scientific Challenges 11

1-6. Practical Applications 12

1-7. Chapter Summary 12

Chapter One Theoretical Framework 13

2-1. Preface 14

2-2. Cryptographic Protocols and End-to-End Data Protection 14

2-3. Data Integrity and Authentication Mechanisms 14

2-4. Threat Models and Adversarial Attacks 15

2-5. Real-Time Communication and N-Tier Architectures 15

2-6. Chapter Summary 18

Abbreviations List

E2EE	**End-to-End Encryption**
HMAC	**Hash-based Message Authentication Code**
IDS	**Intrusion Detection System**
AES	**Advanced Encryption Standard**
MITM	**Man-In-The-Middle**
IDPS	**Intrusion Detection and Prevention System**
LSTM	**Long Short-Term Memory**
HIDS	**Host-based Intrusion Detection System**
DTO	**Data Transfer Object**
RSA	**Rivest-Shamir-Adleman**
IoC	**Indicators of Compromise**
HTTPS	**Hypertext Transfer Protocol Secure**
TLS	**Transport Layer Security**
SQL	**Structured Query Language**

Introduction

1-1. Preface

Artificial Intelligence Engineering and modern information technologies play a very important role in shaping our digital world. They help automate complex tasks, improve decision-making, and build smart software systems for everyone. Inside this broad field, Systems Security and Computer Networks are extremely important. The success of any digital system depends on its ability to protect data and secure communication channels against cyber threats.

Today, real-time messaging applications are essential for personal and business communication. People use them not only for text messages but also to share images, videos, audio, and documents. However, relying heavily on digital communication brings serious security risks. Hackers can intercept data, change messages, or steal information from databases. Because of this, we need strong security designs.

Modern software engineering must combine fast, real-time communication with strict data protection. This project focuses on designing and developing a secure real-time chat application. The application uses a React frontend and an ASP.NET Core backend, relying on SignalR for fast two-way communication. To keep user data completely private, the system uses End-to-End Encryption (E2EE). This means messages and files are never decrypted while traveling across the network. Additionally, the system checks that data is original and unchanged by using cryptographic hashing and Hash-based Message Authentication Codes (HMAC) combined with symmetric encryption.

1-2. Scientific Problem and Project Justifications

A major challenge in network security today is protecting user privacy and ensuring data is not changed during transmission. When users send sensitive text messages, images, videos, audio, and documents over a network, they face threats like eavesdropping, unauthorized changes to messages, fake identities, and database leaks.

The scientific problem this project solves is how to guarantee complete privacy, data integrity, and authentic communication for real-time messages and multimedia files. The project is important because there is a great need for a strong security model that protects data at every step.

To solve this problem, the project uses strict End-to-End Encryption (E2EE) directly from the sender's device to the receiver's device. This ensures that messages and files are never decrypted while traveling across the network or passing through the server. Moreover, to stop hackers from changing or faking messages, the system uses hashing and HMAC with symmetric encryption. This confirms the data's integrity and the sender's true identity. Finally, stored records in the server's database are protected using Advanced Encryption Standard (AES) encryption.

1-3. Project Objectives

The main goal of this project is to design and build a complete, real-time secure communication platform using a clean N-Tier architecture. The platform supports sharing text and multimedia while using strict End-to-End Encryption and HMAC authentication.

Specifically, we aim to build a smooth user interface using React and a strong backend using C# ASP.NET Core. The system is divided into clear layers (Presentation, Business Logic, and Data Access) to make it easy to manage and scale. We also use SignalR technology to provide fast, two-way communication for sending text, images, videos, audio, and documents with live delivery updates.

To ensure complete privacy, our project uses hybrid End-to-End Encryption. We use AES-256 for encrypting the data and RSA-2048 for securely exchanging keys. This means data is encrypted on the sender's device and decrypted only on the receiver's device. The server never reads the data. Additionally, we verify all messages and files in real time using cryptographic hashing, HMAC, and symmetric encryption. We use secure HTTPS/TLS protocols for the network and protect the SQL Server database using AES encryption for stored data and Bcrypt for hashing user passwords safely.

1-4. Previous Studies and State-of-the-Art

In the first study, titled "A Secure Messaging Platform with Advanced Protection Against MITM Attacks and Intrusion Detection/Prevention Using Machine Learning" (2025), Dr. S. Bala Priya, Baratam Sai Anjan Kumar, and Paidi Akhil developed the FortiChat platform to protect digital communications from interception and unauthorized access. The researchers implemented End-to-End Encryption (E2EE) with dynamic session keys so that only the intended recipients can read the transmitted messages, and they integrated an Intrusion Detection and Prevention System (IDPS) that uses rule-based signature detection to identify known attack patterns, inspect incoming and outgoing traffic for malicious payloads, and quarantine dangerous files immediately. In relation to our project, this study directly supports our architectural integration of End-to-End Encryption with a rule-based Intrusion Detection System (Snort IDS) to inspect shared images, videos, audio clips, and documents against known virus signatures and block infected files before they reach the user.

In the second study, titled "AI-Based Cyber Security Intrusion Detection System" (2026), Aman et al. designed and implemented a real-time web-based chat environment capable of handling text messages, images, videos, and file uploads while actively monitoring the communication channel for security threats. The authors built a testing environment to simulate Man-in-the-Middle (MITM) attacks, unauthorized message modifications, and malware file transfers, demonstrating how real-time monitoring and integrity checking can detect tampering and notify users and

administrators through an interactive security alert system. In relation to our project, we adopt the concept of supporting full multimedia file sharing (images, videos, audio, and documents) inside a real-time chat environment while enforcing strict message integrity verification and generating instant security alerts whenever Snort IDS or HMAC verification detects a malicious file or tampered payload.

In the third study, titled "NetAgent: Multi-Task Agentic Network Traffic Analysis Made Practical" (2026), Fu et al. addressed the challenges of practical network traffic analysis and intrusion detection across complex data streams. The researchers reviewed classic deep packet inspection and signature-based intrusion detection tools such as Snort and Suricata, which match network packets and file payloads against predefined rule sets with very low false positive rates, and organized traffic inspection into structured functional layers to handle diverse threat types efficiently. In relation to our project, this study highlights the reliability and precision of signature-based Snort IDS rules for detecting known malware and virus patterns in file transfers, which justifies our deployment of Snort at both the sender and receiver endpoints to inspect unencrypted files before encryption and after decryption.

In the fourth study, titled "Impact of Benign Connectivity Variations on Intrusion Detection for Encrypted OPC UA Traffic in Industrial Private 5G Networks" (2026), Ha et al. investigated how intrusion detection systems operate in communication environments that rely heavily on encrypted traffic. The study demonstrated that while payload encryption protects data confidentiality in transit, it prevents traditional network-based IDS sensors located in the middle of the network from reading the encrypted payload, and showed that normal connection changes such as automatic session reconnects can cause false alarms if not properly handled. In relation to our project, since our chat system applies strict End-to-End Encryption where messages and files are never decrypted in transit, this study justifies placing the Snort IDS directly at the sender and receiver endpoints rather than on the intermediate network, while properly managing SignalR WebSocket reconnections.

In the fifth study, titled "Security Analysis of Agentic AI Communication Protocols: A Comparative Evaluation" (2025), Yedidel Louck, Ariel Stulman, and Amit Dvir evaluated the security of modern communication protocols across five core domains: authentication, authorization, data integrity, confidentiality, and availability. The researchers analyzed protocols that exchange multimodal messages containing text and binary files using MIME-typed multipart structures, and proved that relying only on transport-layer encryption without mandatory per-message cryptographic verification leaves systems vulnerable to Man-in-the-Middle tampering and replay attacks. In relation to our project, we follow the authors' recommendation by combining secure transport channels (HTTPS/TLS and SignalR) with mandatory per-message integrity and authentication checks using cryptographic hashing, HMAC, and symmetric encryption for every text message, image, video, audio file, and document.

In the sixth study, titled "Encrypted Communication with Intelligent Threat Detection: A Secure Chat Framework" (2025), Granthaalayah. developed a secure messaging architecture designed to protect organizational communications against both external database breaches and internal account compromises. The researchers implemented Advanced Encryption Standard (AES-256) encryption to protect stored message payloads, salted Bcrypt cryptographic hashing to secure user passwords against brute-force and rainbow table attacks, and a real-time monitoring module that combines Long Short-Term Memory (LSTM) networks with a Kalman Filter to smooth noisy communication streams and reduce false-positive security alerts. In relation to our project, we adopt this study's foundational database security mechanisms by applying AES-256 Data-at-Rest encryption with a protected master key alongside salted and peppered Bcrypt password hashing, while ensuring our endpoint intrusion detection operates with minimal false alarms during live SignalR sessions.

In the seventh study, titled "SIDiLDNG: A Similarity-Based Intrusion Detection System Using Improved Levenshtein Distance and N-Gram for CAN" (2024), Jiaru Song and Guihe Qin proposed a real-time Intrusion Detection System designed to detect message spoofing, replay, and unauthorized injection attacks in sequential communication streams. The authors categorized communication identifiers and combined N-gram sequence rules with an improved Levenshtein distance algorithm inside a hybrid block-and-stream inspection model, achieving high detection accuracy and fast real-time processing against malicious traffic injections. In relation to our project, this research supports our real-time stream inspection methodology, demonstrating the importance of verifying sequential message integrity and inspecting incoming communication payloads before they are processed by the receiving endpoint.

In the eighth study, titled "Towards a Better Similarity Algorithm for Host-Based Intrusion Detection System" (2023), Ouarda Lounis, Malika Bourenane, and Brahim Bouderah designed a Host-Based Intrusion Detection System (HIDS) deployed directly on endpoint hosts to detect malicious execution traces and harmful payloads. By monitoring system-level call sequences at the host level and comparing incoming data sequences against predefined behavioral and signature rules, the authors demonstrated that endpoint-based inspection can accurately identify malicious files and attack patterns that network-level firewalls might miss. In relation to our project, this study directly supports our architectural decision to deploy the Snort Intrusion Detection System (IDS) at both the sender and receiver host endpoints, enabling the system to scan images, videos, audio clips, and documents locally for viruses where the files are in their unencrypted state.

In the ninth study, titled "Injecting Falsehoods: Adversarial Man-in-the-Middle Attacks Undermining Factual Recall in LLMs" (2025), Alina Fastowski, Bardh Prenkaj, Yuxiao Li, and Gjergji Kasneci introduced the Xmera adversarial Man-in-the-Middle (MITM) attack framework. The researchers demonstrated how an active attacker positioned between a client and a server can intercept user transmissions in real time and subtly alter the payload by injecting misleading instructions, false context, or random noise without the user noticing, and they proposed a defensive detection mechanism to alert users whenever a transmission has been manipulated. In

relation to our project, this study highlights the critical danger of in-transit payload modification and directly justifies our implementation of strict End-to-End Encryption (AES-256 and RSA-2048) combined with per-message HMAC authentication and symmetric encryption, ensuring that any Man-in-the-Middle attempt to alter a text message or media file in transit is immediately detected and rejected.

In the tenth study, titled "FortiChat: Secure Messaging Solution" (2025), the authors presented a comprehensive secure messaging architecture designed to protect personal and professional digital communications across text, audio, and video channels from Man-in-the-Middle (MITM) attacks and unauthorized access. The study implemented End-to-End Encryption (E2EE) alongside dynamic key exchange protocols that generate distinct encryption keys for every session, while also encrypting message metadata such as timestamps, sender, and recipient details to prevent data exposure in transit and at rest. Furthermore, the platform integrated a real-time Intrusion Detection and Prevention System (IDPS) with an automated alarm and response mechanism that utilizes rule-based signature detection to scan incoming and outgoing traffic for malicious payloads and immediately quarantine dangerous files without disrupting active user sessions. In relation to our project, this study directly supports our implementation of session-based End-to-End Encryption (AES-256 and RSA-2048) combined with HMAC authentication and symmetric encryption to protect message and file integrity, as well as our deployment of the signature-based Snort IDS at the sender and receiver endpoints to detect and block viruses hidden inside shared images, videos, audio clips, and documents.

Table 1-1: Studies Comparison

Paper Title	Author	Publisher	Year	Relation to the Project
AI-Based Cyber Security Intrusion Detection System	Aman et al	International Journal / Academic Press	2026	Supports building a real-time chat environment that handles text, images, videos, and files while detecting MITM tampering and alerting users to malware threats
NetAgent: Multi-Task Agentic Network Traffic Analysis Made Practical	Fu et al	IEEE / arXiv	2026	Highlights the high accuracy and low false-positive rate of signature-based Snort IDS rule

				matching for inspecting network traffic and file payloads
Impact of Benign Connectivity Variations on Intrusion Detection for Encrypted OPC UA Traffic in Industrial Private 5G Networks	Ha et al	IEEE / Academic Press	2026	Explains why encrypted traffic cannot be inspected by mid-network sensors, justifying our placement of Snort IDS at the sender and receiver endpoints alongside E2EE
Paper Title	Author	Publisher	Year	Relation to the Project
Encrypted Communication with Intelligent Threat Detection: A Secure Chat Framework	Granthaalayah	Granthaalayah	2025	Behavioral anomaly detection, Firebase database, real time synchronization, smart algorithms
SIDiLDNG: A similarity-based IDS using improved Levenshtein Distance	J. Song et al	Computers & Security	2024	Sequential similarity analysis, pattern extraction, high detection and classification accuracy
Towards a better similarity algorithm for host-based IDS	O. Bourenane	De Gruyter	2023	Detection accuracy up to 99%, strong real-time performance, practical application of the algorithm

FortiChat: Secure Messaging Solution	Scribd Preprint	Scribd	2025	ML algorithms for classification, advanced encryption, real time protection and alerting
Injecting Falsehoods: Adversarial MITM Attacks	arXiv	arXiv	2025	Framework for directed MITM attacks, text manipulation, and interception
A Secure Messaging Platform with Advanced Protection Against MITM Attacks and Intrusion Detection Using Machine Learning	Dr. S. Bala Priya, Baratam Sai Anjan Kumar, and Paidi Akhil	IARJSET	2025	Supports combining End-to-End Encryption (E2EE) with a rule-based Intrusion Detection System to detect known attack signatures and malicious files in real-time chat
Security Analysis of Agentic AI Communication Protocols: A Comparative Evaluation	Yedidel Louck, Ariel Stulman, and Amit Dvir	arXiv (Ariel University)	2025	Demonstrates the necessity of combining secure transport with mandatory per-message integrity verification (such as HMAC) for multimodal text and binary file transfers

1-5. Scientific Challenges

Developing a complete secure chat system with real-time multimedia sharing, End-to-End Encryption, and HMAC authentication brings several scientific and technical challenges. One main challenge is mixing real-time connections (SignalR) with standard web architecture (HTTP) to keep users connected and update message delivery statuses without losing data. Another challenge is building the system using the strict N-Tier architecture, which requires moving Data Transfer Objects (DTOs) safely across the Presentation, Business Logic, and Data Access layers without showing the internal database structure. Finally, executing strict client-side End-to-End Encryption (RSA and AES) alongside HMAC hashing for every text, image, video, and document without slowing down the fast real-time chat experience required careful programming and optimization.

1-6. Practical Applications

The security methods designed in this project have important practical applications for businesses, governments, and places that need high privacy. Using direct End-to-End Encryption alongside server Data-at-Rest encryption makes this platform perfect for company chats where employees share private text messages, financial documents, images, and videos. This protects sensitive company data from network hackers and database theft. Furthermore, the platform is great for sectors that need guaranteed data authenticity and strict regulatory compliance, like healthcare telemedicine, legal offices, and banking services. In these fields, verifying the sender's identity and making sure files have not been changed in transit (using HMAC) are essential requirements.

1-7. Chapter Summary

This chapter provided an introduction to the project, starting with the importance of Systems Security and Computer Networks in protecting modern digital communication. We defined the scientific problem of keeping real-time text and media safe from eavesdropping, message tampering, and database leaks. We explained our Defense-in-Depth architecture, which is built on End-to-End Encryption, HMAC integrity and authentication, and Data-at-Rest database encryption. The chapter also summarized ten recent state-of-the-art studies from 2024 to 2026, comparing their findings. Finally, we discussed the scientific challenges and the practical applications of the proposed platform.

Chapter One
Theoretical Framework

2-1. Preface

This chapter builds the theoretical foundation needed to design a secure real-time communication system. It explains the mathematical and structural ideas behind modern security protocols, especially End-to-End Encryption (E2EE) and protecting data stored in databases (Data-at-Rest). It also explains how to ensure that messages and multimedia files are authentic and unchanged during transmission by using cryptographic hashing, Hash-based Message Authentication Codes (HMAC), and symmetric encryption. Additionally, it covers how to secure user passwords using salted BCrypt hashing. Finally, this chapter looks at modern cyber threats, such as Man-in-the-Middle (MITM) attacks and replay attacks, and explains the real-time N-Tier software architecture used to build the system.

2-2. Cryptographic Protocols and End-to-End Data Protection

Cryptography is the science of securing digital information using math, ensuring that only the right people can read the data. For chat applications that handle sensitive text messages, images, videos, audio, and documents, keeping data private is very important. To achieve this, our project uses End-to-End Encryption (E2EE). In the E2EE model, every message and file is encrypted on the sender's device. It stays completely encrypted while crossing the network and the server, and is only decrypted on the receiver's device, meaning no hacker or server administrator can view the private conversations. To make this fast and secure, the system uses a mix of two encryption types. First, the system uses the Advanced Encryption Standard (AES-256) to encrypt the actual data. AES is a very fast and strong symmetric encryption method that uses a new session key for every chat exchange. To safely share this AES key between the sender and receiver without exposing it, the system uses the Rivest-Shamir-Adleman (RSA-2048) asymmetric algorithm. Every user device creates a pair of keys, consisting of a Public Key shared openly to encrypt the session key, and a Private Key kept secretly on the device to decrypt the session key. Besides protecting data while it travels, we must also protect it when it is saved. The system uses Data-at-Rest encryption with a secure server master key to encrypt all saved payloads in the SQL Server database. This ensures that if the database is ever stolen, the records remain entirely unreadable.

2-3. Data Integrity and Authentication Mechanisms

Data integrity means proving that information remains accurate and has not been changed during transmission or storage. In a secure chat, encryption hides the data, but it does not stop a hacker from changing the encrypted bits while they are traveling. To fix this, the system uses a Hash-based Message Authentication Code (HMAC) along with symmetric encryption for every message and file. Before sending a message or file, the sender's device creates a unique HMAC code based on the data and a secret key. The receiver's device receives the data and calculates its own HMAC

code. If both codes match exactly, the system mathematically proves data integrity, meaning not a single byte of the text or media was changed during travel. It also proves message authentication, confirming that the message truly came from the verified sender rather than an impersonator. Furthermore, user passwords must be strongly protected in the database. Passwords are never saved as normal text. Instead, the system uses the BCrypt hashing function, which turns passwords into irreversible math values. By adding a unique random text called a salt to every password before hashing it, the system effectively defeats hacking methods like brute-force and dictionary attacks.

2-4. Threat Models and Adversarial Attacks

To secure a chat platform, we must understand the threats it faces. A major threat is the Man-in-the-Middle (MITM) attack, in which a hacker secretly sits between two users to listen to or change the data passing through the network. Without E2EE and HMAC verification, a MITM attacker could read private chats, alter files, or redirect communication flows without anyone knowing. Another dangerous threat is the replay attack, where a hacker captures a legitimate encrypted message or login token while it is traveling, and tries to send it again later to trick the system into repeating an unauthorized action. By combining strict E2EE, unique session keys, and HMAC, the system actively defends against these interception and tampering attacks.

2-5. Real-Time Communication and N-Tier Architectures

Building a fast and secure messaging platform requires a clean software structure known as an N-Tier architecture. This architecture logically divides the system into three separate layers to ensure maintainability and separation of concerns. The Presentation Layer, built with React, handles the user interface, local encryption and decryption, HMAC generation, and displaying media. The Business Logic Layer, built with C# ASP.NET Core, manages user sessions, routes the messages, and enforces security rules, using Data Transfer Objects (DTOs) to move data cleanly without exposing the underlying database structure. The Data Access Layer handles secure interactions with the SQL Server database, enforcing the AES Data-at-Rest encryption and Bcrypt password hashing. To make communication instant across this structure, the system uses Microsoft SignalR technology. SignalR creates a continuous, two-way connection between the React client and the backend. This allows the instant transfer of encrypted text, images, videos, audio, and documents, along with live delivery updates and instant security alerts if HMAC detects any tampering.

This figure below will show how the system works:

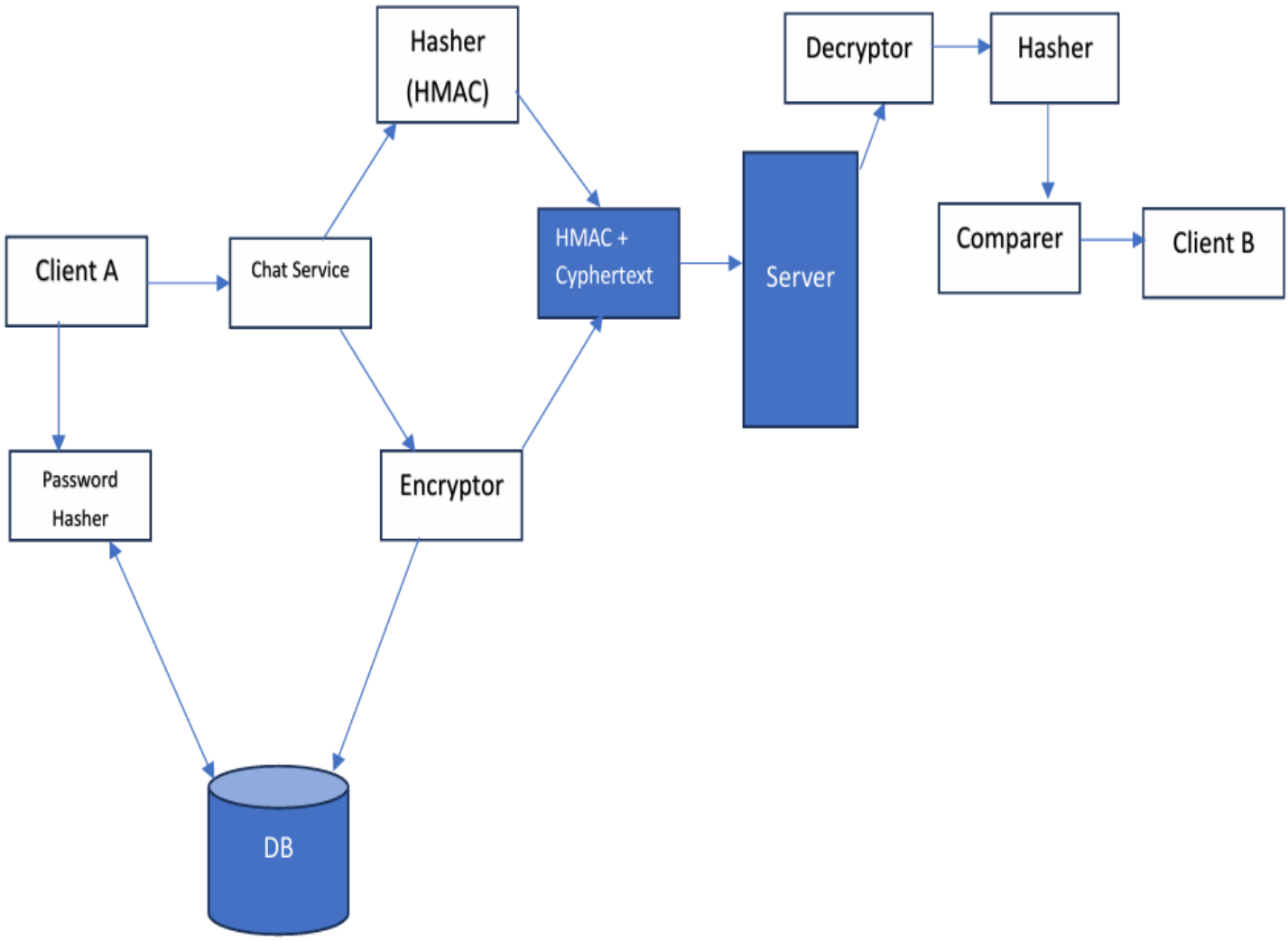


Figure 1-1: Block Diagram

This figure below will show the flow-chart of the system flow:

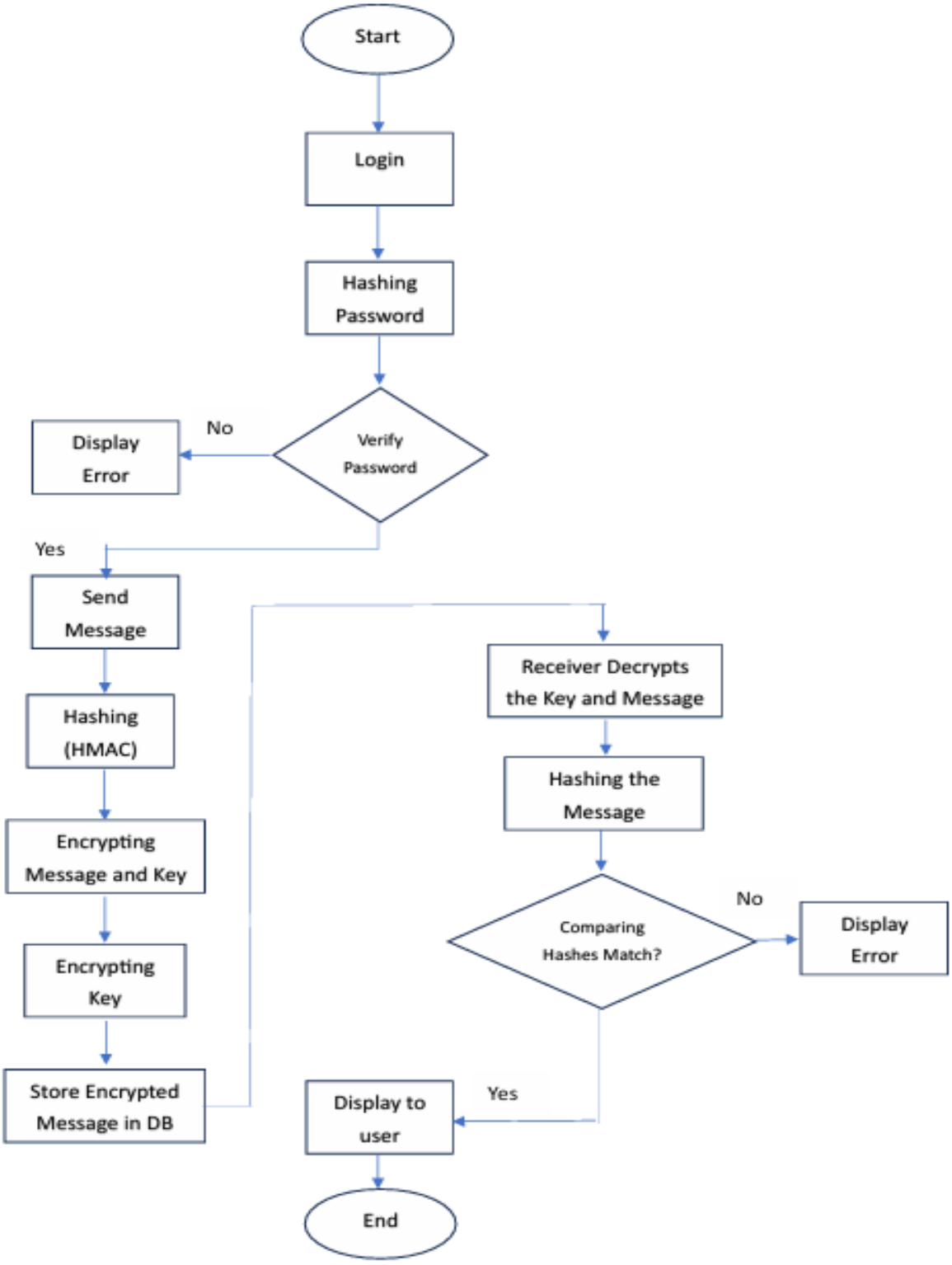


Figure 1-2: Flow-Chart

2-6. Chapter Summary

This chapter reviewed the theoretical foundations and security protocols used in our platform. It explained how End-to-End Encryption using AES-256 and RSA-2048 keeps messages and media completely private during transmission, and how Data-at-Rest encryption protects the database. It also showed how data integrity and sender identity are guaranteed for every message using HMAC and symmetric encryption, along with BCrypt for passwords. Finally, the chapter discussed modern network threats like MITM and replay attacks, and explained the real-time SignalR and N-Tier software architecture used to build the system safely and efficiently.